Rapport de Projet : Migration et Gouvernance des Données

Cas d'étude n°3 : Milieu Hospitalier

Auteur :
Eliot KONDRYC

Formation :
B3 ASRBD
Campus de Lille

20 mars 2026

Table des matières

1 Introduction **2**

2 Environnement Technique **2**

3 Infrastructure de Données **2**
3.1 Conteneurisation (Docker Compose) 2

4 Logique de Migration et Traitements **2**
4.1 Génération du Dataset (`generate_data.py`) 2
4.2 Processus de Migration (`migration.py`) 2
4.3 Reprise d'anomalies (`reprise.py`) 3

5 Audit et Validation (`validate.py`) **3**

6 Explication des scripts de migration et gestion des données **3**
6.1 Génération des données (`generate_data.py`) 3
6.2 Migration et Nettoyage (`migration.py`) 3
6.3 Validation Post-Migration (`validate.py`) 4
6.4 Reprise des anomalies (`reprise.py`) 4

7 Conclusion **5**

1 Introduction

Ce document présente l'architecture, les scripts et les procédures mis en place pour le projet de gestion, nettoyage, et migration des données hospitalières. Ce projet s'inscrit dans une démarche de fiabilisation des données critiques.

2 Environnement Technique

L'ensemble du projet a été développé sous un environnement moderne garantissant la portabilité et la répétabilité des tests.

- **Langage** : Python 3.13
- **Base de données** : MariaDB (Conteneurisée via Docker)
- **Bibliothèques clés** :
  - `pandas` : Analyse et transformation massive.
  - `pymysql` : Interface de communication SQL.
  - `faker` : Mocking de données pour les tests de charge.

3 Infrastructure de Données

3.1 Conteneurisation (Docker Compose)

Au lieu d'une installation locale polluante pour le système hôte, une approche par **conteneurisation** a été privilégiée. Le service utilise une image `mariadb:10.11`.

Commande de déploiement : `docker compose up -d`

Le port `3306` est exposé pour permettre aux scripts Python d'interagir directement avec l'instance de production ou de test.

4 Logique de Migration et Traitements

4.1 Génération du Dataset (`generate_data.py`)

Ce module assure la création d'un jeu de test cohérent.

- **Règles métiers :** Unicité des numéros de série (format `MT-XXXXX`), exclusion des dates d'intervention futures.
- **Traçabilité :** Un fichier `log.txt` est généré systématiquement pour valider l'intégrité des fichiers CSV produits.

4.2 Processus de Migration (`migration.py`)

C'est le cœur du système de transfert. Il opère une fusion intelligente entre différentes sources (`v1` et `v2`) tout en normalisant les noms de colonnes. Les doublons sont filtrés via `Pandas` avant l'injection SQL.

4.3 Reprise d'anomalies (`reprise.py`)

Ce script fait office de **file d'attente de quarantaine**. Il nettoie les données corrompues (prix négatifs, formats de dates erronés) et isole les erreurs fatales dans une table spécifique pour expertise humaine.

5 🔍 Audit et Validation (`validate.py`)

La phase finale consiste en un audit automatisé vérifiant :

1. La cohérence volumétrique entre CSV et SQL.
2. L'absence de doublons au niveau des contraintes de clés primaires.
3. L'intégrité référentielle (pas d'interventions "orphelines").

6 Explication des scripts de migration et gestion des données

Le projet s'articule autour de quatre scripts Python principaux, chacun ayant un rôle précis dans le cycle de vie, la migration et la validation des données au sein de la base de données MariaDB (`cas3_hopital`).

6.1 Génération des données (`generate_data.py`)

Ce script est responsable de la création du jeu de données initial simulant l'environnement de l'hôpital en utilisant des bibliothèques poussées comme `Faker` et `pandas`.

Voici un exemple illustrant la génération aléatoire automatisée pour les établissements :

```python
def generate_etablissements(num_rows):
    data = []
    types_etab = ['Clinique', 'Centre médical', 'CHU', 'EHPAD']
    for i in range(1, num_rows + 1):
        data.append({
            "id_etablissement": i,
            "nom": fake.company() + " Santé",
            "adresse": fake.street_address(),
            "ville": fake.city(),
            "type": random.choice(types_etab)
        })
    df = pd.DataFrame(data)
    df.to_csv("data_3/etablissements.csv", index=False)
```

6.2 Migration et Nettoyage (`migration.py`)

C'est le script principal d'intégration des données dans la base de production. Avant l'insertion en BDD, on effectue une fusion et un nettoyage des données (ETL).

Extrait montrant la gestion des équipements (fusion des listes V1 et V2 et suppression des doublons sur l'ID) :

```python
def process_equipements(cursor, filepath_v1, filepath_v2):
    df1 = pd.read_csv(filepath_v1)
    df2 = pd.read_csv(filepath_v2)

    # Transformation V2 pour avoir le même format que la V1
    mapping = {"ref": "id_equipement", "designation": "type", "fabricant": "marque"}
    df2_transformed = df2.rename(columns=mapping)

    # Fusion des deux datasets et suppression des doublons
    df = pd.concat([df1, df2_transformed], ignore_index=True)
    df = df.drop_duplicates(subset=['id_equipement'])

    # ... on vérifie l'intégrité et on insère en BDD ...
```

6.3 Validation Post-Migration (validate.py)

Ce script intervient après la migration pour s'assurer de la qualité et de la cohérence des données intégrées en exécutant des requêtes SQL.

Voici comment on automatise la vérification de l'intégrité référentielle (pour s'assurer qu'il n'y a aucun orphelin) :

```python
    queries_relations = [
        ("Equipements -> Etablissements",
         "SELECT COUNT(*) FROM equipements e LEFT JOIN etablissements et ON e.id_etablissement = et.id_etablissement WHERE et.id_etablissement IS NULL"),
        # ...
    ]

    for label, query in queries_relations:
        orphelins = fetch_single_value(cursor, query)
        statut = "OK" if orphelins == 0 else f"VIOLATION ({orphelins} enregistrements)"
```

6.4 Reprise des anomalies (reprise.py)

Ce script est conçu pour corriger spécifiquement le fichier contenant des erreurs en testant et modifiant activement les champs de manière autonome.

Voici le cas pratique où on corrige automatiquement un prix de manière conditionnelle :

```python
        try:
            prix_float = float(prix)
            if prix_float < 0:
                # CORRECTION : on ramène le prix négatif en prix positif (erreur de signe probable)
                final_prix = abs(prix_float)
                is_corrected = True
            elif prix_float == 0:
                erreurs.append("Le prix ne peut être égal à 0")
```

```
        except ValueError:
            erreurs.append(f"Prix non numérique ({prix})")
```

7 Conclusion

Ayant un parcours en **BTS SIO option SLAM**, la logique algorithmique et la manipulation de bases de données sont des concepts que je maîtrise depuis deux ans. Ce projet a permis de mettre en application ces compétences dans un contexte de gouvernance de données plus rigoureux.